Université Mohammed V — Rabat
Master SNIA — Big Data

SE3 — Pôle ETL & Batch

Sprint 2 — ETL Complet MOCK

Étudiantes : Abbar Amina & Nader Chaimaa

Encadrant : Pr. EL ASRI Mohamed Youssef

1. Contexte et objectif du Sprint 2

Le Sprint 2 s'inscrit dans le pipeline Big Data du projet Yelp Academic Business. Il fait suite au Sprint 1 (nettoyage local avec Pandas) et constitue la phase d'ETL complet sur le dataset MOCK, en utilisantPySpark et Spark SQL.

L'objectif principal est de prendre les lignes propres issues de S1 et de les enrichir via des transformations distribuées, pour produire un Data Warehouse structuré au format Parquet, exploitable par SE4 (streaming) et SE5 (visualisation).

Flux du pipeline : `clean_S1.csv` ⟶ **Sprint 2 ETL (PySpark)** ⟶ `clean_S2.parquet` + `5 agrégats Parquet`

2. Technologies utilisées

2.1 PySpark

PySpark est l'API Python d'Apache Spark. Il permet de traiter des volumes massifs de données en mode distribué sur un cluster. Chaque transformation s'applique en parallèle sur des partitions du DataFrame, ce qui le rend adapté au Big Data.

2.2 Spark SQL

Spark SQL permet d'exécuter des requêtes SQL classiques directement sur un DataFrame Spark, via une vue temporaire créée avec `createOrReplaceTempView()`. Il est utilisé dans le Groupe 3 pour les cinq agrégations analytiques.

```
# Creation de la vue temporaire
df.createOrReplaceTempView("businesses")

# Requete SQL classique sur le DataFrame
avg_stars = spark.sql("""
    SELECT city, ROUND(AVG(stars), 2) AS avg_stars
    FROM businesses
    GROUP BY city
    ORDER BY avg_stars DESC
""")
```

Listing 1 – Exemple d'utilisation de Spark SQL

2.3 Format Parquet

Parquet est un format de stockage colonnaire binaire. Contrairement au CSV qui stocke les données ligne par ligne, Parquet les organise par colonne, permettant une lecture sélective très rapide.

Critère	CSV	Parquet
Format	Texte lisible	Binaire colonnaire
Taille	Grande	Compressée (5×)
Lecture rapide	Non	Oui (10×)
Compatible Spark	Oui	Oui (natif)
Ouvrir dans Excel	Oui	Non

2.4 Stack technique complète

Technologie	Version	Rôle dans S2
Python	3.14	Langage principal
PySpark	3.x	Moteur de traitement distribué
Spark SQL	3.x	Agrégations analytiques
Java	17 (Temurin)	Runtime Spark
Parquet + Snappy	—	Format de sortie colonnaire compressé
winutils	hadoop-3.3.5	Compatibilité Windows

3. Données d'entrée

Le Sprint 2 reçoit en entrée le fichier CSV nettoyé produit par S1 :

Paramètre	Valeur
Fichier source	`clean_S1.csv`
Nombre de lignes	54 (dataset MOCK)
Colonnes	`business_id, name, city, state, stars, review_count, categories, is_open, latitude, longitude`

4. Transformations réalisées

4.1 Groupe 1 — Formatage et Standardisation

Ces transformations corrigent les types et le format des données existantes sans créer de nouvelles colonnes.

#	Transformation	Colonne	Justification
T1	`int` → `boolean`	`is_open`	0/1 sémantiquement incorrect ; boolean plus explicite et filtrage plus rapide
T2	Remplacer `&` par `and`	`categories`	Évite les conflits Spark SQL et Superset
T3	Arrondir à 1 décimale	`stars`	Lisibilité et cohérence entre datasets
T4	Supprimer espaces superflus	`categories`	Mesure préventive pour le Sprint 3 (données réelles)

4.2 Groupe 2 — Enrichissement Métier

Ces transformations créent 8 nouvelles colonnes calculées à partir des données existantes, apportant de la valeur analytique au Data Warehouse.

#	Colonne	Calcul	Type
T5	`categories_list`	`split(categories, ', ')`	Array[String]
T6	`nb_categories`	`size(categories_list)`	Integer
T7	`score_class`	`stars<2→Faible / 2-4→Moyen / 4→Bon`	String
T8	`region`	Mapping `state` → zone géographique USA/Canada	String
T9	`is_restaurant`	`array_contains(list, 'Restaurants')`	Boolean
T10	`is_healthcare`	`array_contains(list, 'Health and Medical')`	Boolean
T11	`name_length`	`length(name)`	Integer
T12	`state_upper`	`upper(state)`	String

4.3 Groupe 3 — Agrégations Analytiques (Spark SQL)

Ces cinq agrégations sont réalisées via Spark SQL. Le DataFrame est d'abord enregistré comme vue temporaire, puis des requêtes SQL classiques sont exécutées et les résultats exportés individuellement en Parquet.

#	Fichier Parquet	Description	KPI
AGG1	`agg_avg_stars_city`	Moyenne des étoiles par ville	Quelle ville a les meilleurs commerces ?
AGG2	`agg_businesses_state`	Distribution par état + région	Où sont concentrés les commerces ?
AGG3	`agg_top_categories`	Top 30 catégories	Quelles catégories dominent ?
AGG4	`agg_score_distribution`	% Bon/-Moyen/Faible par région	Quelle région a la meilleure qualité ?
AGG5	`agg_segmentation`	Restaurant vs Healthcare vs autres	Quel segment est le mieux noté ?

4.4 Groupe 4 — Export Parquet

Tous les résultats sont exportés en **Parquet** (format columnar compressé via Snappy).

Avantage	Impact
5× moins lourd que CSV	Réduction du stockage
10× plus rapide à lire par Spark	Gain en performance
Compatible HDFS, Druid, Superset	Interopérabilité SE4 / SE5

5. Structure du projet

```
S2/
          data/
```

```
                clean_S1.csv                        # Entree
nettoyee Sprint 1
                clean_S2.parquet/                   # Dataset principal
    enrichi (8 colonnes)
                agg_avg_stars_city.parquet/         # AGG1 : moyenne stars
     par ville
                agg_businesses_state.parquet/       # AGG2 : distribution
    par etat
                agg_top_categories.parquet/         # AGG3 : top 30
    categories
                agg_score_distribution.parquet/ # AGG4 : score_class
    par region
                agg_segmentation.parquet/       # AGG5 : segmentation
    metier
        sprint2_etl.py                         # Script principal
    PySpark
        README.md
```

Listing 2 – Arborescence du répertoire S2

6. Perspectives — Sprint 3

Le Sprint 3 appliquera ce même pipeline sur les **vraies données Yelp Academic** (plusieurs millions de lignes) en environnement **HDFS**. Les adaptations prévues sont :

— Remplacement du chemin local par un chemin HDFS (`hdfs://...`)

— Remplacement de local[*] par l'URL du cluster Spark

— Activation de la gestion des partitions pour optimiser les performances